Introduction to Machine Learning Paradigms

7th Annual Deep Learning Indabax Uganda 2026

Lukman Bukenya

March 2026

The start and the Future

About Me and What I Do

AI Research Scientist and Software Engineer
Geospatial Data and Computational Intelligence Lab, Makerere University

-
-
-

Protein Design
Learning biological structure and function

3D/4D Vision
Spatial-temporal understanding of complex data

Research and Community Building

Research identity

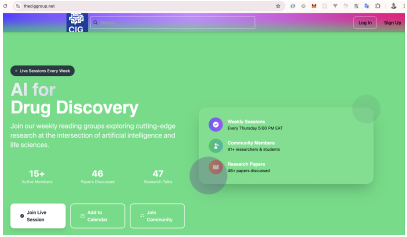
I am an **AI Research Scientist and Software Engineer** at the **Geospatial Data and Computational Intelligence Lab, Makerere University**. My work is motivated by:

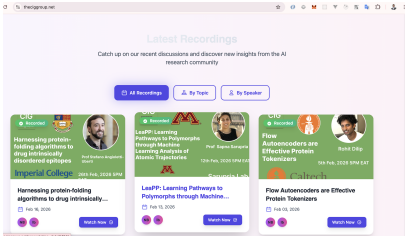
The desire to be part of the great scientifc community that is pushing boundaries in AI and biology?

Our Reading Group

I am also a **co-host of the Computational Intelligence Group (CIG)**, a community reading group where we discuss recent work in:

- AI for drug discovery
- protein modeling and design

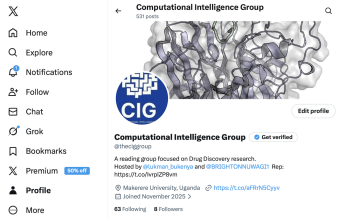


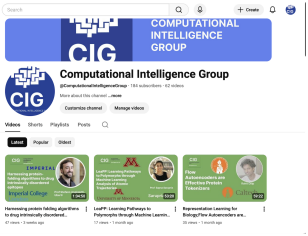


The Computational Intelligence Group (CIG)

Our style of engagement

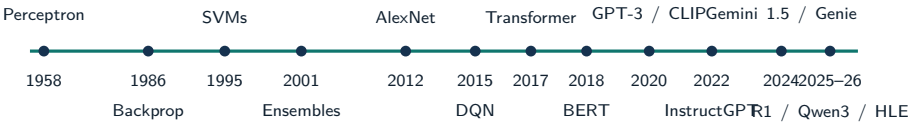
We learn together, ask questions, and asking how these ideas can influence future work in biology and medicine.



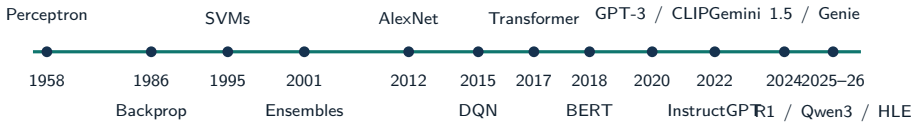


Checkout our platforms live sessions every week, recorded talks on youtube, and our website at www.theciggroup.net

Where we have come from

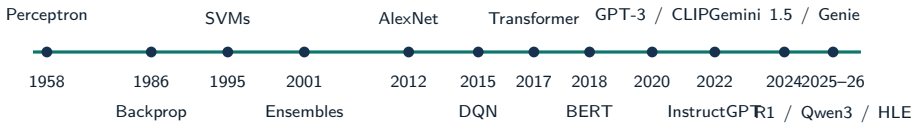


Where we have come from



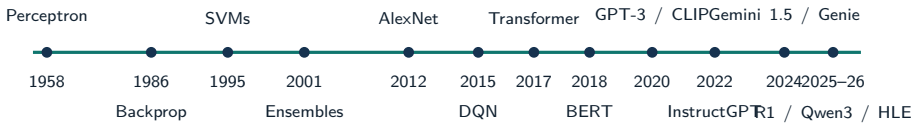
- In the early 20th century, Andrey Markov introduced the concept of random values and created a theory of stochastic processes. We know them in artificial intelligence (AI) as Markov Decision Processes (MDPs), Markov Chains, and Markov Processes

Where we have come from



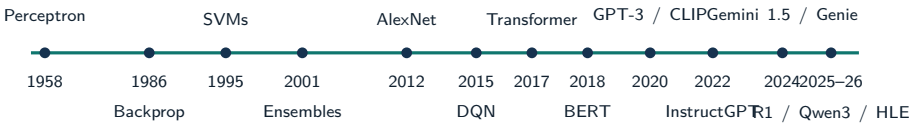
- In 1902, Markov showed that we could predict the next element of a chain, a sequence, using only the last past element of that chain. In 1913, he applied this to a 20,000-letter dataset using past sequences to predict the future letters of a chain. Bear in mind that he had no computer but managed to prove his theory, which is still in use today in AI(Sequence Prediction).

Where we have come from



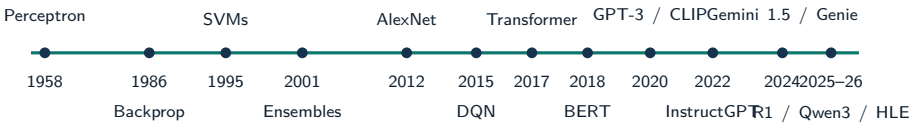
- In 1948, Claude Shannon's *The Mathematical Theory of Communication* was published. He cites Andrey Markov's theory multiple times when building his probabilistic approach to sequence modeling. Claude Shannon laid the ground for a communication model based on a source encoder, a transmitter, and a received decoder or semantic decoder.

Where we have come from



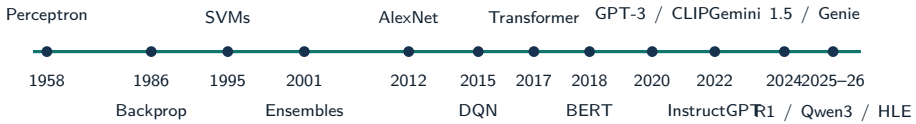
- In 1950, Alan Turing published his seminal article: *Computing Machinery and Intelligence*. Alan Turing based this article on machine intelligence on the immensely successful Turing Machine that decrypted German messages. The expression artificial intelligence was first used by John McCarthy in 1956. However, Alan Turing was implementing artificial intelligence in the 1940s to decode encrypted encoded messages in German.

Where we have come from



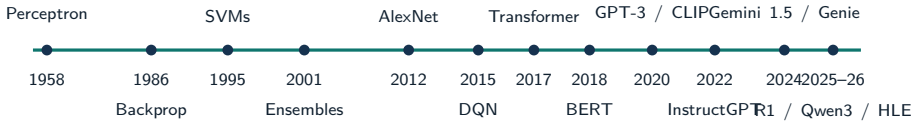
- In 1954, the Georgetown-IBM experiment used computers to translate Russian sentences into English using a rule system. A rule system is a program that runs a list of rules that will analyze language structures. Rule systems still exist. However, creating rule lists for the billions of language combinations in our digital world is a challenge yet to be met. For the moment, it seems impossible. But who knows what will happen?

Where we have come from



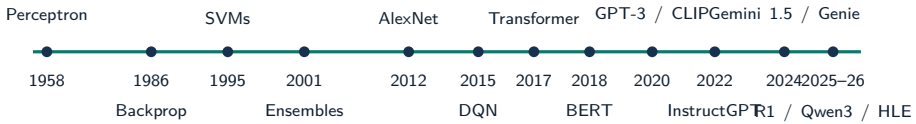
- In 1982, John Hopfield introduced Recurrent Neural Networks (RNNs), known as Hopfield networks or "associative" neural networks. John Hopfield was inspired by W.A. Little, who wrote The Existence of Persistent States in the Brain in 1974. RNNs evolved, and LSTMs emerged as we know them. An RNN memorizes the persistent states of a sequence efficiently:

Where we have come from



- In the 1980s, Yann Le Cun designed the multi-purpose Convolutional Neural Network (CNN). He applied CNNs to text sequences, and they have been widely used for sequence transduction and modeling as well. They are also based on persistent states that gather information layer by layer. In the 1990s, summing up several years of work, Yann Le Cun produced LeNet-5, which led to the many CNN models we know today. The CNN's otherwise efficient architecture faces problems when dealing with long-term dependencies in very long and complex sequences.

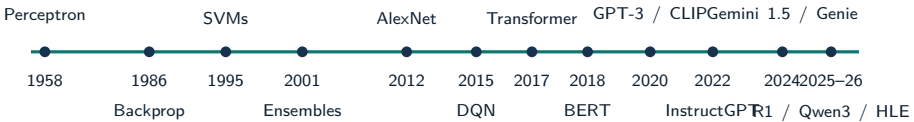
Where we have come from



The field keeps moving the **locus of learning**.

- In early machine learning, we mostly learned decision boundaries on top of features we engineered.

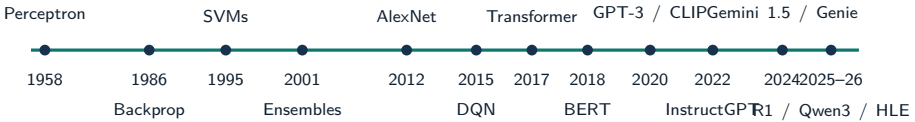
Where we have come from



The field keeps moving the locus of learning.

- In early machine learning, we mostly learned decision boundaries on top of features we engineered.
- In deep learning, we learned the representations themselves from raw data.

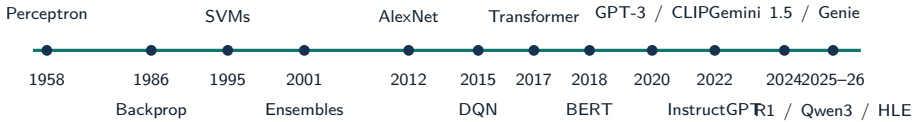
Where we have come from



The field keeps moving the locus of learning.

- In early machine learning, we mostly learned decision boundaries on top of features we engineered.
- In deep learning, we learned the representations themselves from raw data.
- In foundation models, we learned a general prior once and reused it across many tasks through prompting and fine-tuning.

Where we have come from



The field keeps moving the locus of learning.

- In early machine learning, we mostly learned decision boundaries on top of features we engineered.
- In deep learning, we learned the representations themselves from raw data.
- In foundation models, we learned a general prior once and reused it across many tasks through prompting and fine-tuning.
- Current frontier work seeks to ask better questions: can models keep learning after deployment?

Where we have come from

The field keeps moving the locus of learning.

- In early machine learning, we mostly learned decision boundaries on top of features we engineered.
- In deep learning, we learned the representations themselves from raw data.
- In foundation models, we learned a general prior once and reused it across many tasks through prompting and fine-tuning.
- Current frontier work asks a better question: can models keep learning after deployment?

So the mental model

1. Hand-crafted features
2. End-to-end representation learning
3. Self-supervised foundation models
4. Test-time adaptation and continual learning

Where we have come from

The field keeps moving the locus of learning.

- In early machine learning, we mostly learned decision boundaries on top of features we engineered.
- In deep learning, we learned the representations themselves from raw data.
- In foundation models, we learned a general prior once and reused it across many tasks through prompting and fine-tuning.
- Current frontier work asks a better question: can models keep learning after deployment?

So the mental model

1. Hand-crafted features
2. End-to-end representation learning
3. Self-supervised foundation models
4. Test-time adaptation and continual learning

Core tension

The more general our models become, the more we care about memory, adaptation, and how learning happens at inference time.

But what counts as a *paradigm*?

A paradigm is more than a model class

It is a bundled answer to five questions:

- What is the learning signal? Labels, structure, reward, or self-generated feedback?

But what counts as a *paradigm*?

A paradigm is more than a model class

It is a bundled answer to five questions:

- What is the learning signal? Labels, structure, reward, or self-generated feedback?
- What is the unit of experience? Examples, trajectories, prompts, or open-ended interaction?

But what counts as a *paradigm*?

A paradigm is more than a model class

It is a bundled answer to five questions:

- What is the learning signal? Labels, structure, reward, or self-generated feedback?
- What is the unit of experience? Examples, trajectories, prompts, or open-ended interaction?
- What kind of memory does the system keep?
- When do parameters change: during training only, or also at test time?

But what counts as a *paradigm*?

A paradigm is more than a model class

It is a bundled answer to five questions:

- What is the learning signal? Labels, structure, reward, or self-generated feedback?
- What is the unit of experience? Examples, trajectories, prompts, or open-ended interaction?
- What kind of memory does the system keep?
- When do parameters change: during training only, or also at test time?
- How is compute split across training and inference?

But what counts as a *paradigm*?

A paradigm is more than a model class

It is a bundled answer to five questions:

- What is the learning signal? Labels, structure, reward, or self-generated feedback?
- What is the unit of experience? Examples, trajectories, prompts, or open-ended interaction?
- What kind of memory does the system keep?
- When do parameters change: during training only, or also at test time?
- How is compute split across training and inference?

And here is why

Two systems can use the same architecture and still belong to different paradigms if they differ in:

But what counts as a *paradigm*?

A paradigm is more than a model class

It is a bundled answer to five questions:

- What is the learning signal? Labels, structure, reward, or self-generated feedback?
- What is the unit of experience? Examples, trajectories, prompts, or open-ended interaction?
- What kind of memory does the system keep?
- When do parameters change: during training only, or also at test time?
- How is compute split across training and inference?

And here is why

Two systems can use the same architecture and still belong to different paradigms if they differ in:

- supervision,

But what counts as a *paradigm*?

A paradigm is more than a model class

It is a bundled answer to five questions:

- What is the learning signal? Labels, structure, reward, or self-generated feedback?
- What is the unit of experience? Examples, trajectories, prompts, or open-ended interaction?
- What kind of memory does the system keep?
- When do parameters change: during training only, or also at test time?
- How is compute split across training and inference?

And here is why

Two systems can use the same architecture and still belong to different paradigms if they differ in:

- supervision,
- memory,

But what counts as a *paradigm*?

A paradigm is more than a model class

It is a bundled answer to five questions:

- What is the learning signal? Labels, structure, reward, or self-generated feedback?
- What is the unit of experience? Examples, trajectories, prompts, or open-ended interaction?
- What kind of memory does the system keep?
- When do parameters change: during training only, or also at test time?
- How is compute split across training and inference?

And here is why

Two systems can use the same architecture and still belong to different paradigms if they differ in:

- supervision,
- memory,
- adaptation schedule,

But what counts as a *paradigm*?

A paradigm is more than a model class

It is a bundled answer to five questions:

- What is the learning signal? Labels, structure, reward, or self-generated feedback?
- What is the unit of experience? Examples, trajectories, prompts, or open-ended interaction?
- What kind of memory does the system keep?
- When do parameters change: during training only, or also at test time?
- How is compute split across training and inference?

And here is why

Two systems can use the same architecture and still belong to different paradigms if they differ in:

- supervision,
- memory,
- adaptation schedule,
- and deployment loop.

But what counts as a *paradigm*?

A paradigm is more than a model class

It is a bundled answer to five questions:

- What is the learning signal? Labels, structure, reward, or self-generated feedback?
- What is the unit of experience? Examples, trajectories, prompts, or open-ended interaction?
- What kind of memory does the system keep?
- When do parameters change: during training only, or also at test time?
- How is compute split across training and inference?

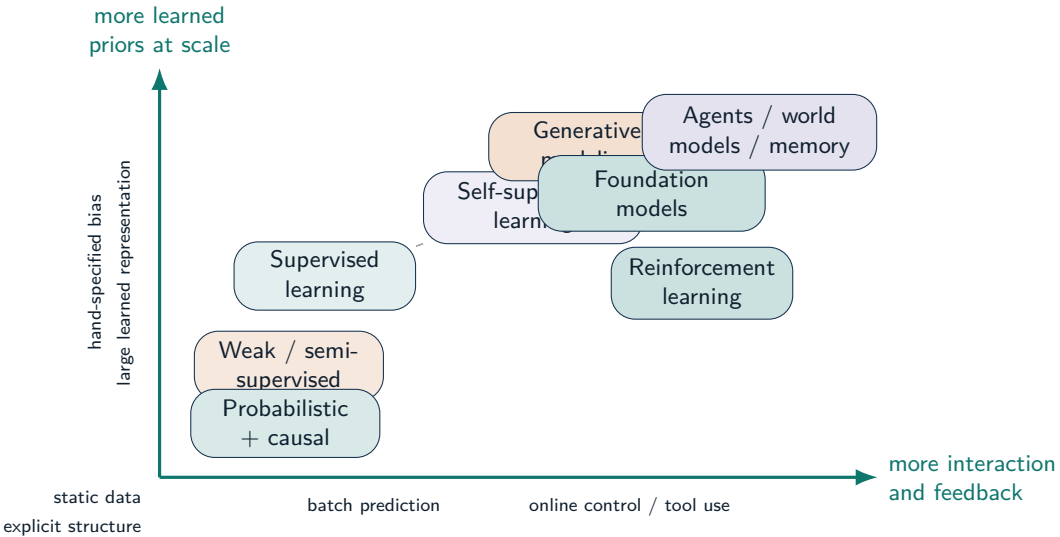
And here is why

Two systems can use the same architecture and still belong to different paradigms if they differ in:

- supervision,
- memory,
- adaptation schedule,
- and deployment loop.

This is why supervised learning, reinforcement learning, foundation models, and test-time learning are powerful paradigms beyond mere implementation details.

A look at Paradigms



Supervised learning => The First Receipe

- Fit parameters to labeled examples by minimizing prediction error.

Supervised learning => The First Receipe

- Fit parameters to labeled examples by minimizing prediction error.
- This paradigm made ML measurable, benchmarkable, and ultimately industrializable.

Supervised learning => The First Receipe

- Fit parameters to labeled examples by minimizing prediction error.
- This paradigm made ML measurable, benchmarkable, and ultimately industrializable.
- It won spectacularly in vision, speech, and later NLP once data and compute became abundant.

Supervised learning => The First Receipe

- Fit parameters to labeled examples by minimizing prediction error.
- This paradigm made ML measurable, benchmarkable, and ultimately industrializable.
- It won spectacularly in vision, speech, and later NLP once data and compute became abundant.
- But it remained limited by label cost, shortcut learning, and distribution shift.

Supervised learning => The First Receipe

- Fit parameters to labeled examples by minimizing prediction error.
- This paradigm made ML measurable, benchmarkable, and ultimately industrializable.
- It won spectacularly in vision, speech, and later NLP once data and compute became abundant.
- But it remained limited by label cost, shortcut learning, and distribution shift.

What supervised learning normalized

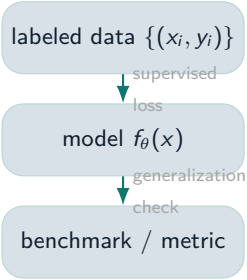
train validation test splits, benchmark culture, hyperparameter search, model selection by held-out performance

Supervised learning => The First Receipe

- Fit parameters to labeled examples by minimizing prediction error.
- This paradigm made ML measurable, benchmarkable, and ultimately industrializable.
- It won spectacularly in vision, speech, and later NLP once data and compute became abundant.
- But it remained limited by label cost, shortcut learning, and distribution shift.

What supervised learning normalized

train , validation , test splits, benchmark culture, hyperparameter search, model selection by held-out performance



$$\min_\theta \sum_i \ell(f_\theta(x_i), y_i)$$

Generative models went from synthesis to structure

VAE
latent variables
smooth representations

→

GAN
adversarial training
sharp samples

→

Diffusion
stable denoising
scalable generation

→

DiT / RAE
transformerized latents
better encoders

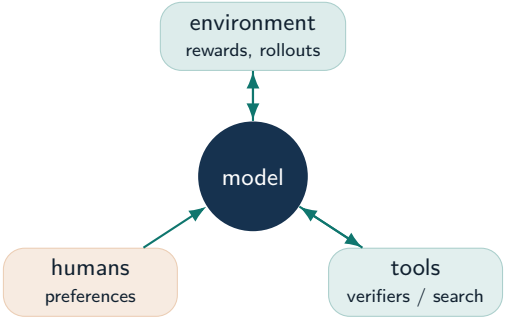
Generative models: Mapping from one distribution to another generative objectives now serve as representation learners, compressors, simulators, and sometimes as the substrate for world models and planning.

Reinforcement and feedback learning closed the loop

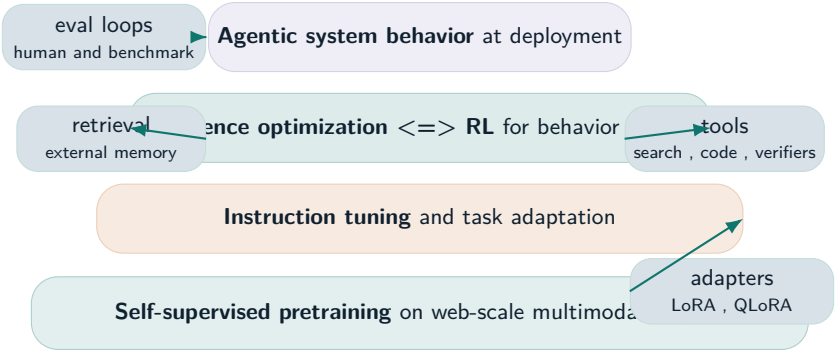
- Reinforcement learning made sequential decision-making central: actions change future data.
- Human feedback and preference optimization made behavior shaping practical for foundation models.
- Recent reasoning systems increasingly treat inference as adaptive compute: search, reflection, verification, and selective deliberation.

The big shift

Models stopped being only predictors and started becoming **actors**: they plan, try, observe, revise, and sometimes call tools.



Foundation models merged paradigms

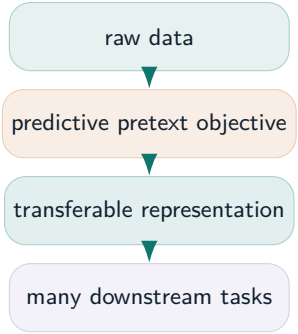


Self-supervision changed data source perspective

- The dominant question changed from *"How do we get more labels?"* to *"What predictive signal is already inside raw data?"*
- Pretraining made representation quality a function of data scale, objective design, and adaptation strategy.
- The result: transfer became the default, and task-specific architecture choice became less central.

What changed?

Once unlabeled data became the main fuel, ML could absorb the web, multimodal corpora, and eventually deployment traces were possible.



Examples
masked prediction
contrastive learning
multimodal alignment

Supervised statistical learning

- The canonical setup was i.i.d. examples with explicit labels and a fixed prediction target.
- Learning meant minimizing empirical risk under a chosen hypothesis class. Linear models, trees, kernels, probabilistic classifiers.
- The inductive bias came from the model family and from human-designed features.
- This paradigm remains extremely strong for tabular data, diagnostics, ranking, and low-data settings.

What it got right

- clear objectives
- sample efficiency
- interpretability in many settings
- strong theory around generalization

What it struggled with

The feature extractor and the learner were too separate. For images, speech, text, and biology, this became a bottleneck.

Unsupervised/Self-supervised and RL: structure and interaction

Learning from structure

Unsupervised / self-supervised learning

- uses data without human labels,
- discovers clusters, latent variables, or predictive structure,
- later becomes the engine of large-scale pretraining.

Useful supervision can often be manufactured from the data itself.

Learning from interaction

Reinforcement learning

- learns from reward instead of labels,
- optimizes over sequences of actions,
- foregrounds exploration, planning, and delayed credit assignment.

This is the paradigm that turns prediction into decision making.

Motivational Take!

Modern foundation models inherit the first; agentic systems and embodied AI inherit the second.

Deep learning changed the bargain

- Backpropagation through deep networks made representation learning the default rather than the exception.
- Convolutional, recurrent, and later attention-based models reduced the need for hand-crafted features.
- The field discovered a powerful recipe., that is, more data, more compute, better optimization, deeper representations.
- This paradigm unlocked breakthroughs not just in perception, but in science(protein design and synthesis) and control.

Representative wins

- ImageNet-scale visual recognition
- end-to-end speech and language modeling
- deep RL for control and games
- protein structure prediction at scientific scale

The conceptual shift

The model no longer sits *after* feature engineering; it absorbs feature engineering into learning.

Birth of Transformer and self-supervised pretraining = foundation model paradigm

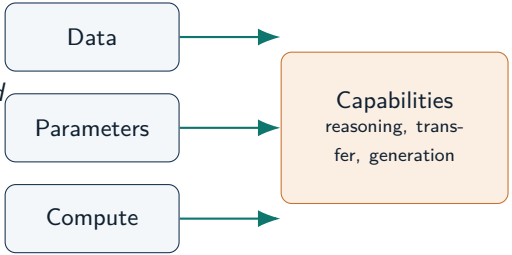
- The Transformer made it practical to train large sequence models with broad context integration.
- Self-supervised objectives turned raw web-scale text, images, audio, and code into training signal.
- Pretraining produced models that could be reused, prompted, and aligned for many downstream tasks.
- The result is a new abstraction: one general model, many interfaces.

The modern stack

Massive corpora	self-supervised objectives
Large model	prompting , finetuning
Instruction tuning	tools , retrieval , APIs
Multimodality	deployment as a platform

Scaling became a paradigm in its own right

- Once architectures and objectives were good enough, scale itself became predictive of capability.
- Research shifted from *can this work?* to *how should we allocate model size, data, and compute?*
- This reframed machine learning as both a scientific and a systems problem.



The practical consequence

Pretraining became industrial infrastructure. The training run itself became an object of strategy, optimization, and capital allocation.

Where we are now

- The dominant paradigm today is the foundation model stack: pretrain broadly, align, optionally fine-tune, then extend with retrieval, tools, and post-training.
- This stack gives us remarkable breadth: drafting, coding, retrieval, multimodal interpretation, and zero-/few-shot transfer.
- But most deployed systems are still static after training. They can use context, but they do not reliably turn new experience into persistent knowledge.

A useful summary

Today's systems are excellent at:

- compressing large training distributions,
- reusing priors in context,
- composing with tools and search.

They are much less mature at:

- continual learning,
- safe self-modification,
- persistent memory across episodes.

Why the current paradigm feels incomplete

Context is expensive

Attention remains powerful, but long-context reasoning is still costly and not always persistent.

Memory is shallow

A prompt can adapt behavior for one session without becoming stable knowledge.

Inference is becoming a learning site

Research is shifting compute and adaptation *into* deployment via retrieval, search, recurrent depth, and test-time updates.

Systems are hybridizing

Modern applications now mix model weights, external memory, tools, and planners.

The theory lag

We still lack clean abstractions for when information should live in weights, context, external memory, or adaptive state.

Frontier A: memory-efficient sequence modeling

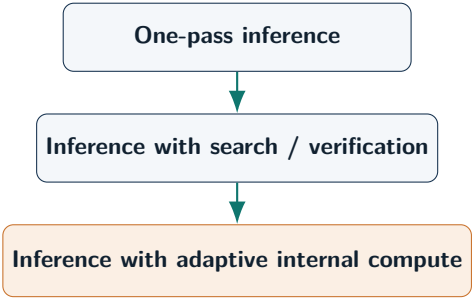
Family	Core idea	What it buys
Transformers	attend directly to rich context	best-in-class retrieval from the visible prompt, but quadratic cost
State-space / recurrent models	compress context into a learned state (for example, Mamba)	linear or near-linear scaling, attractive for long sequences
Memory modules that learn at test time	update a hidden state by optimization (for example, TTT layers, Titans)	more adaptive hidden state; a step toward learned memory rather than static recall

Why this line matters

The field is no longer asking only *how can attention be made cheaper?* It is also asking *what should a model remember, compress, or update while it reads?*

Frontier B: more compute at test time

- Reasoning performance is increasingly improved by spending compute after training: sampling, verification, search, re-ranking, or recurrent refinement.
- This blurs a once-clean line: training was where learning happened; inference was where the model merely executed.
- Recent work even explores latent-space recurrence, where extra thinking happens inside the hidden state rather than only in generated tokens.



Paradigm signal

Capability is no longer purchased only with more parameters; it can also be purchased *per query*.

Frontier C: retrieval, tools, and adaptation

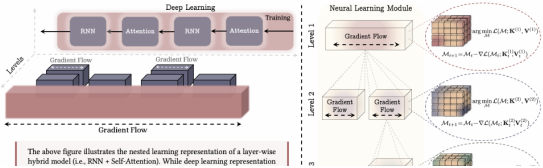
External memory	Test-time learning	Agents and interaction
Retrieval-augmented systems keep knowledge outside the model and fetch it when needed.	Recent work adapts weights or low-rank parameters online to distribution shift and fresh data.	The reinforcement learning tradition is reappearing inside language-model systems via planning, tool use, and closed-loop action.

The big idea

The future may not belong to a single mechanism. It may belong to compositions of pretrained priors, retrieval, adaptive state, and interactive learning.

Spotlight from Nested Learning: The Illusion of Deep Learning Architecture

- Behrouz, Razaviyayn, Zhong, and Mirrokni propose a provocative reframing: treat a machine learning system as a set of nested optimization problems with different update frequencies.
- On this view, architecture and optimizer are not separate species; they are parts of one interconnected learning module.
- This lens tries to unify pretraining, in-context learning, optimizer state, and continual learning into one story about how context gets compressed over time scales.



Adapted from Figure 2 in (**behrouz2025nested**).

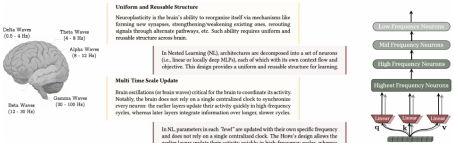
Why I think Nested Learning is a big deal

Three memorable claims from the paper

1. Popular optimizers can be interpreted as memory systems over gradients.
2. Pretraining can be viewed as a form of in-context learning at massive scale.
3. Better continual learning may require multiple learning time scales, not just deeper static networks.

HOPE and continuum memory

The paper's proposed direction is a multi-timescale memory architecture intended to improve persistent learning and long-context behavior.



Adapted from Figure 1 in (**Behrouz et al**).

Nested Learning: The Illusion of Deep Learning Architecture
What remains open

Why it is valuable

- It gives a strong vocabulary for multi-timescale learning.
- It connects naturally to other 2024–2025 threads TTT, Titans, recurrent depth, and test-time learning.
- It turns continual learning from an add-on into a first-class design question.

Open scientific questions

- How stable are self-modifying systems at scale?
- Which memories should persist, and for how long?
- How do we benchmark adaptation without rewarding unsafe drift?
- What is the systems cost of richer memory and online updates?

My read

Nested Learning is best understood as a research program rather than a settled replacement for the Transformer paradigm.

Where the field is likely heading

Move	Why we should care
Bigger *and* better-trained priors	scale will remain important, but smarter data and compute allocation matter as much as raw size
Longer-lived memory	systems will need ways to keep useful information beyond a single prompt window
More adaptive inference	search, verification, recurrent depth, and online updates will buy capability at deployment time
Hybrid architectures	retrieval, tools, recurrent state, and static parameters will be composed rather than treated as rivals
Interactive learning loops	agents, planning, and continual experience will bring reinforcement learning ideas back to center stage

A possible synthesis: from *trained models* to *learning systems*

Pretrained prior → Retrieval / tools → Adaptive hidden state → Persistent memory / continual updat

The emerging design question

For any capability, where should the information live?

- in fixed weights,
- in retrieved context,
- in a transient adaptive state,
- or in persistent memory accumulated over time?

Discussion prompts for the room

Research questions

1. When should a model update context, state, or weights?
2. What benchmark actually measures persistent learning rather than mere prompt fitting?
3. Can we build adaptive systems that stay auditable and safe?

Design questions

1. Is the future one giant universal model, or a modular learning system?
2. How much learning should happen at deployment time?
3. What should be forgotten on purpose?

The next paradigm(if not already happening) may be about how models learn in deployment, not only what they learned before deployment.

Some Take-aways

1. Machine learning paradigms have shifted the location of learning: from handcrafted features, to learned representations, to large pretrained priors, and now toward adaptive inference and continual memory.
2. Foundation models define the present, but current research is reopening questions about memory, recurrence, test-time compute, and persistent adaptation.
3. Nested Learning is useful because it reframes the debate. Instead of asking only *which architecture wins*, it asks which parts of the system should update, at what time scale, and on what context.

Thank you.

Sources: (**behrouz2025nested**)